ChartBreak

Visual Diagnosis of Where LLMs and Prompting Strategies Fail at Chart Generation

Team 11 DS-GA 3001 Final Project Proposal

1 Project Title & Team Information

Title: ChartBreak: Visual Diagnosis of Where LLMs and Prompting Strategies Fail at Chart Generation.
Team 11: Shubham Vijay Mojidra (sm13126@nyu.edu), generation harness, prompting strategies, and D3 interface; Hyun Sang Ryu (hr2712@nyu.edu), evaluation pipeline and statistics; Rahil Shaikh (rss9385@nyu.edu), open-model inference on NYU HPC and the failure predictor. The user study, interpretation, and writing are shared.

Problem and motivation. Large language models (LLMs) are now routinely asked to turn a sentence such as "show the number of students advised by each faculty rank as a pie chart" into plotting code. The VisEval benchmark [1] shows that this still fails often: with one fixed prompt, GPT-4 produced a correct Matplotlib chart for 75.3% of queries and CodeLlama-7B for 28.2%, and quality dropped for multi-channel and harder queries. Aggregate pass rates tell a developer *that* a model fails, but not *which* queries fail, *at which step* (code that crashes, wrong data transformation, wrong chart type, wrong sort order), or *which prompting change fixes which failure and breaks what else*. We will build a visual analytics tool that answers those questions for several LLMs and prompting strategies, using ground-truth checks rather than human judgment of every chart.

2 Research Paper & Critical Analysis

Key paper: VisEval [1] (Chen et al., IEEE VIS 2024 Best Paper). *Data.* The authors clean nvBench [2], whose labels contain errors and ambiguous queries, through rules, a majority vote of three LLMs, and expert review. The result has 2,524 natural-language queries over 146 databases, describing 1,150 charts of 7 types, about 2.19 paraphrases per chart, and four hardness levels. Ground truth is a *feasible region*: chart type, plotted data, which channels the query fixes, and whether sorting is required, so several correct charts are accepted. *Method.* Generated code is checked in three stages. A *validity* checker runs it in a sandbox; a *legality* checker deconstructs the rendered SVG and checks chart type, data, and sort order separately; a *readability* checker uses GPT-4V to rate layout and scales (agreement with experts SRCC 0.843). *Findings.* Using one prompting method (CoML4VIS, one example), GPT-4, GPT-3.5, Gemini-Pro, and CodeLlama-7B pass 75.3%, 61.8%, 51.6%, and 28.2% with Matplotlib. Switching GPT-4 to Seaborn raised invalid code from 3.3% to 25.4%, and adding two irrelevant tables cut its pass rate to 65.9%. Errors fall into five categories: invalid code, illegal data transformation, illegal visual transformation, illegal sorting, and low readability.

Critical analysis. VisEval's main contribution is reliable, automatic, stage-wise checking, which turns chart generation into a task with per-query labels; its toolkit and data are open (MIT). Four points limit what it tells a developer. (i) Results are aggregate tables plus hand-picked example failures, so there is no way to see which queries fail together or how a failure changes between conditions. (ii) Prompting is held fixed: each model is tested with one method, and alternatives (Chat2VIS [3], LIDA [4]) are compared only on 689 single-table charts with GPT-3.5. (iii) Paraphrases of the same chart are counted as independent queries, which overstates the effective sample size. (iv) The models tested are now dated, and readability depends on GPT-4V.

What we build on. We reuse VisEval's data, checkers, and agent interface unchanged and treat its stage-wise outcomes as per-query labels. We add prompting strategy as a controlled factor, report uncertainty clustered by chart rather than by paraphrase, and build linked views for per-query diagnosis. From Manifold [7] we borrow the idea of comparing two models instance by instance and selecting where they disagree, applied here to pairs of prompting strategies.

Related work. Chat2VIS [3] shows prompt engineering for natural language to visualization on case studies, Prompt4Vis [5] adds example mining and schema filtering and reports gains on nvBench, and LIDA [4] splits generation into summarizing, goal setting, generation, and repair. They propose prompting methods; none shows per-query where a method helps or hurts. ChainForge [6] is a general visual toolkit for comparing prompts and models on response sets, but it has no notion of task-specific failure stages or ground-truth chart checks.

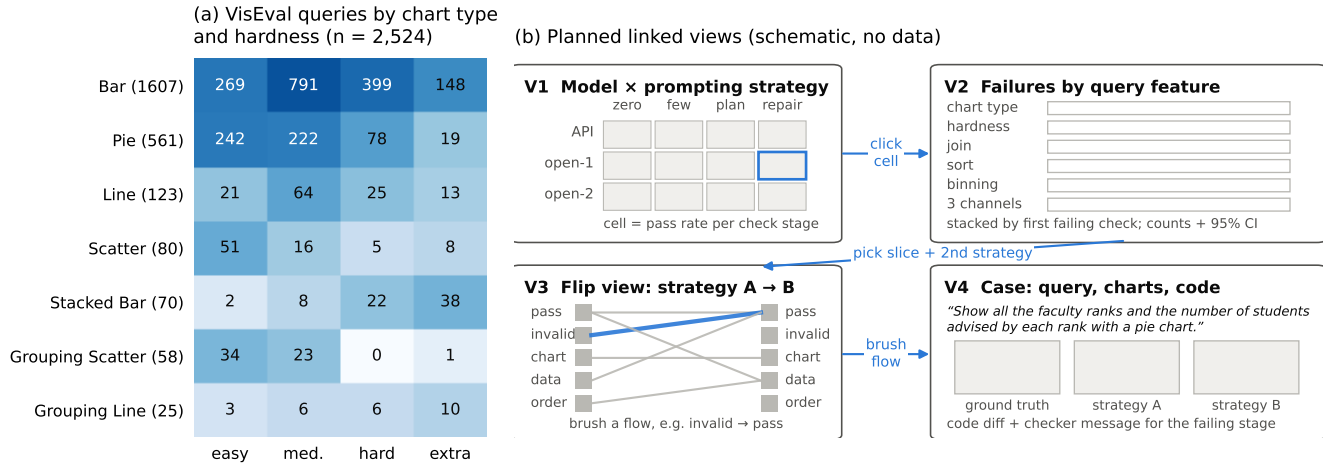


Figure 1: (a) Composition of the VisEval benchmark, computed from the released data: 64% of queries ask for bar charts and some chart-type and hardness cells have fewer than 10 queries, so every view shows counts and intervals instead of bare rates. (b) Schematic of the planned linked views; boxes contain no data.

3 Proposed Project & Innovation

Research questions. RQ1: For current LLMs, at which check stage do chart-generation failures happen, and which query properties (chart type, hardness, joins, sorting, binning, three channels, multiple tables) concentrate them? **RQ2:** How do prompting strategies change the *distribution* of failures, which failures does each one fix, and which new failures does it introduce? **RQ3:** Do linked views, including a per-query flip view, help users find failure patterns more accurately or faster than VisEval's standard outputs?

Hypotheses to test. H1: Self-repair (S4) mostly turns invalid code into passes but rarely fixes wrong data or sort order, because execution errors carry no information about the query. **H2:** Plan-then-code (S3) reduces data and order failures on multi-table and sort-required queries, at some cost in invalid code for the smaller models. **H3:** Few-shot examples (S2) help most on rare chart types (stacked and grouped charts), where we expect zero-shot output to fall back to plain bars. **H4:** Failures are partly predictable from query features (AUROC above the hardness-and-chart-type baseline). Any of these may turn out false; the flip view is designed to show it either way.

How visualization and ML connect. The ML models under study are the LLMs (and a small failure predictor, below); visualization is how we compare and debug them. A pass rate of, say, 70% for two strategies can hide thousands of queries that one strategy fixes and the other breaks; only per-query, linked views show that.

Planned views (Figure 1b). Each step answers one question:

View	User action → what updates	Question answered
V1 Model × strategy	Click a cell (pass rate by check stage, with CI) → V2 filters to that condition	Which model and prompting combination fails most, and at which stage?
V2 Failures by query feature	Pick a slice (e.g. multi-table, sort required) and a second strategy → V3	Which query properties concentrate failures for this condition?
V3 Flip view A → B	Brush a flow between outcome categories (e.g. invalid → pass) → V4 lists those queries	What does strategy B fix relative to A, and what does it break?
V4 Case view	Inspect query, ground-truth chart, both generated charts, code diff, checker message	Why did this query fail under A and not under B?

Rates in every view come with counts and 95% intervals, and small cells are flagged. An optional query map (sentence embeddings projected with UMAP, colored by outcome) supports brushing similar queries.

Failure predictor. To go beyond visual inspection, we fit a logistic regression and a gradient-boosted classifier that predict failure from query features (the RQ1 properties, query length, schema size, number of tables) plus model and strategy, explained with SHAP [8]. Splits are grouped by database so the predictor cannot memorize a schema.

SHAP values describe the predictor, not causes of LLM failure; the predictor's job is to rank slices for V2 and to test how predictable failures are from query features.

Expected contributions. (1) A controlled study of 3 LLMs × 4 prompting strategies on VisEval, with stage-wise outcomes and chart-clustered intervals. (2) A linked D3 tool whose flip view shows, per query, what a prompting change fixes and breaks. (3) A failure predictor with database-grouped evaluation. (4) A small user evaluation against VisEval's standard outputs. Which strategy is best, and whether the tool helps users, are open questions; null or negative results will be reported and explained. The novelty is the per-query, per-stage diagnosis across prompting strategies, not a new prompting method or benchmark.

4 Project Plan & Evaluation

Data (in hand). We downloaded the VisEval data and toolkit from the public repository. Our own count of the released files matches the paper: 1,150 charts, 2,524 queries, 146 databases. Of the queries, 37% need multiple tables, 49% require a sort order, 36% involve a join, 11% need binning, and 6% use a third channel (Figure 1a). The repository already includes CoML4VIS, Chat2VIS, and LIDA agents behind a common interface, which we will extend rather than rewrite.

Pilot: harness check (done). Before spending any LLM calls, we drew every ground-truth chart with a short Matplotlib template (no LLM) and ran VisEval's own validity and legality checkers on all 2,524 queries. 2,457 (97.3%) pass. Of the 67 failures, 43 are order failures because our template plots the stored data order instead of applying the query's sort, and 1 is a template shape error. The other 23 queries (11 pie charts) are correct charts that the data checker rejects: it matches labels by prefix, so when one label is a prefix of another ("PP" and "PPT", "Chevrolet" and "Chevrolet House") it reports the value as found more than once, and it fails to match some long labels. Checker false failures are therefore real but rare; these 23 queries seed the planned audit, and V4 will flag them.

Splits. To avoid tuning on test queries, we hold out 20% of databases as a development pool for prompt design and few-shot examples, and evaluate on the remaining databases (about 2,000 queries).

Models and prompting strategies. One low-cost hosted API model and two open-weight 7–8B instruction or code models (e.g. Qwen2.5-Coder-7B-Instruct, Llama-3.1-8B-Instruct) served with vLLM on NYU HPC, all at temperature 0 with Matplotlib. Strategies: **S1** zero-shot (query, schema, data preview); **S2** few-shot with three examples retrieved from development databases; **S3** plan-then-code, where the model first writes the data transformation and chart specification (type, axes, sort) and then the code; **S4** self-repair, S1 followed by up to two rounds that return execution errors and the model's own self-check. S4 never sees legality-checker output, because that uses ground truth. VisEval's CoML4VIS prompt is run as a reference point. That is about $3 \times 4 \times 2{,}000 = 24{,}000$ generations plus repair calls, which is cheap for the API model and a few GPU-hours per open model.

Metrics and statistics. For each query and condition we record the first failing stage (invalid, illegal chart type, illegal data, illegal order) or pass. Readability, which needs a vision model, is run on a stratified sample of about 300 queries per condition. Pass rates get 95% intervals from a bootstrap that resamples charts, keeping paraphrases together; strategy comparisons use paired per-query outcomes (McNemar tests with Holm correction, with chart-clustered bootstrap as a check). The failure predictor is scored by AUROC and AP under database-grouped 5-fold cross-validation against a baseline that uses hardness and chart type only. Because VisEval marks unparseable charts as illegal by default, two of us will hand-audit 100 random failures to estimate the checker's false-failure rate and report agreement (Cohen's κ).

Visualization evaluation. *Usage scenarios:* two documented walk-throughs, for example tracing what self-repair fixes for multi-table queries and what it breaks. *User study:* 6–8 non-team participants, within-participant, comparing (A) VisEval's standard outputs (aggregate tables plus per-query result logs in a spreadsheet) with (B) our linked tool, with balanced order and matched task sets. Each task has an answer computed from the logs: T1, which strategy fixes the most sort-order failures for a given model; T2, which chart type has the highest failure rate among multi-table queries; T3, how many queries S4 breaks that S1 passed; T4, the failing stage and its cause for a given query. We report accuracy, time, and feedback per participant, descriptively, and limit claims to these tasks and participants.

Schedule and division of labor.

Milestone	Deliverable	Owner
Nov 3 update	Harness running S1–S4 on the API model; first stage-wise results with intervals; V1–V2 in D3	Shubham (harness, D3); Hyun (statistics)
Nov 17	Both open models on HPC; V3–V4 linked; failure predictor; 100-failure checker audit	Rahil (HPC, predictor); Shubham (D3); Hyun and Rahil (audit)
Nov 28	User study sessions; final statistics; usage scenarios written up	Hyun (study design, analysis); all (sessions)
Dec 1/8, Dec 14	10-minute demo; 8-page report	All

The team meets weekly. **Risks:** API cost (capped by the development/evaluation split and temperature 0), HPC queue time (fallback to one open model), and checker errors (measured by the audit). Out of scope: training or fine-tuning models, new benchmarks, and non-Python chart libraries. Git records prompts, model versions, seeds, and AI assistance.

Limitations and success. VisEval covers 7 common chart types in static Matplotlib output and English queries, so results may not transfer to dashboards or other libraries, and model versions change quickly. Its queries derive from the public nvBench, so pretraining contamination may inflate absolute pass rates; our main comparisons are between strategies on the same model, which contamination affects less. Success is a reproducible, statistically careful comparison and a tool that makes per-query failure patterns visible, not a predetermined pass rate.

References

[1] N. Chen, Y. Zhang, J. Xu, K. Ren, and Y. Yang. 2024. VisEval: A Benchmark for Data Visualization in the Era of Large Language Models. *IEEE TVCG* (Proc. IEEE VIS 2024). github.com/microsoft/VisEval.

[2] Y. Luo et al. 2021. Synthesizing Natural Language to Visualization (NL2VIS) Benchmarks from NL2SQL Benchmarks. *ACM SIGMOD*.

[3] P. Maddigan and T. Susnjak. 2023. Chat2VIS: Generating Data Visualizations via Natural Language Using ChatGPT, Codex and GPT-3 Large Language Models. arXiv:2302.02094.

[4] V. Dibia. 2023. LIDA: A Tool for Automatic Generation of Grammar-Agnostic Visualizations and Infographics using Large Language Models. *ACL System Demonstrations*.

[5] S. Li, X. Chen, Y. Song, Y. Song, and C. Zhang. 2024. Prompt4Vis: Prompting Large Language Models with Example Mining and Schema Filtering for Tabular Data Visualization. arXiv:2402.07909.

[6] I. Arawjo, C. Swoopes, P. Vaithilingam, M. Wattenberg, and E. Glassman. 2024. ChainForge: A Visual Toolkit for Prompt Engineering and LLM Hypothesis Testing. *ACM CHI*.

[7] J. Zhang, Y. Wang, P. Molino, L. Li, and D. S. Ebert. 2019. Manifold: A Model-Agnostic Framework for Interpretation and Diagnosis of Machine Learning Models. *IEEE TVCG* 25(1), 364–373.

[8] S. M. Lundberg and S.-I. Lee. 2017. A Unified Approach to Interpreting Model Predictions. *NeurIPS* 30.